B. Choi, *Introduction to Python Network Automation*

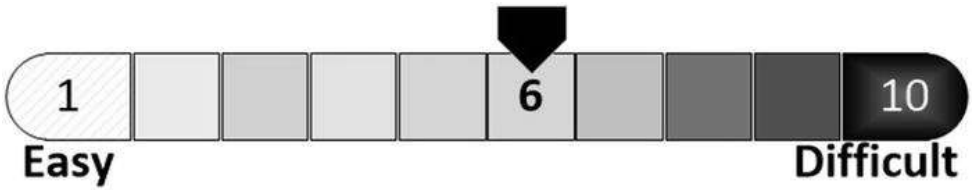
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_8

8. Linux Basic Administration

Brendan Choi[1]

(1) Sydney, NSW, Australia

There are plenty of books on Linux topics, and some of these books focus on only one Linux distribution while being hundreds of pages long. Continuing from the previous chapter, we will be covering the most fundamental Linux subjects in this chapter so that you can get through this book. If you are a newcomer to Linux and want to understand more about the Linux operating system (OS), it is a good idea to use a dedicated Linux book. You will learn to locate Linux system information, use Linux network commands, and then install TFTP, FTP, SFTP, and NTP services on your CentOS 8.1 server.



Information on Linux: Kernel and Distribution Version

Most of the new Linux users come up with this first question, “How can I check the exact Linux distribution version and kernel?” The Linux kernel is the main component of a Linux OS and is the core interface between a computer’s hardware and processes. How do you check the Linux kernel and distribution versions on your OS? The exercises in this chapter are based on CentOS (a Red Hat–based OS), and due to the command syntax of different Linux distributions, some commands used might not work on Debian-based OSs (Ubuntu). If this is the case, then you will be reminded of such differences. However, most of the Linux commands across different Linux distributions are similar.

① How do you check the Linux kernel version? Use the `uname` command with different options; the commands used on CentOS and Ubuntu are the same for `uname` commands .

```
For CentOS 8.1
[root@centos8s1 ~]# uname
Linux
[root@centos8s1 ~]# uname -o
GNU/Linux
[root@centos8s1 ~]# uname -r
4.18.0-193.14.2.el8_2.x86_64
[root@centos8s1 ~]# uname -or
4.18.0-193.14.2.el8_2.x86_64 GNU/Linux
For Ubuntu 20.04 LTS
root@ubuntu20s1:~# uname
Linux
root@ubuntu20s1:~# uname -o
GNU/Linux
root@ubuntu20s1:~# uname -r
5.4.0-47-generic
root@ubuntu20s1:~# uname -or
5.4.0-47-generic GNU/Linux
```

② The following exercise is an additional `uname` command exercise that stores null-terminated operating system information strings into structured references by name. Run the `uname` commands with different options. Note, the same commands can be used on a Ubuntu 20.04 LTS server .

```
[root@centos8s1 ~]# uname -a          # -a option for
all
Linux centos8s1 4.18.0-193.14.2.el8_2.x86_64 #1 SMP
Sun Jul 26 03:54:29 UTC 2020 x86_64 x86_64 x86_64
GNU/Linux
[root@centos8s1 ~]# uname -n          # -n for name or
hostname
centos8s1
[root@centos8s1 ~]# uname -r          # -r for Linux
kernel version
4.18.0-193.14.2.el8_2.x86_64
[root@centos8s1 ~]# uname -m          # -m for system
architecture
x86_64
```

```
[root@centos8s1 ~]# uname -v          # -v for system
time and timezone
#1 SMP Sun Jul 26 03:54:29 UTC 2020
[root@centos8s1 ~]# uname -rnmv          # combined
options
centos8s1 4.18.0-193.14.2.el8_2.x86_64 #1 SMP Sun Jul
26 03:54:29 UTC 2020 x86_64
```

On CentOS and Red Hat–based Linux OSs, you can also use the `grep` command to check the Grub Environment Block. Use the `grep saved_entry /boot/grub2/grubenv` command. This command does not work on Debian-based Linux distributions.

③ For CentOS 8.1

```
[root@centos8s1 ~]# grep saved_entry
/boot/grub2/grubenv
saved_entry=e53c131bb54c4fb1a835f3aa435024e2-4.18.0-
193.14.2.el8_2.x86_64
```

④ Use `hostnamectl` to check the hostname, OS, kernel version, architecture, and more .

```
For CentOS 8.1
[root@centos8s1 ~]# hostnamectl
 Static hostname: centos8s1
   Icon name: computer-vm
     Chassis: vm
   Machine ID: e53c131bb54c4fb1a835f3aa435024e2
     Boot ID: 648950d070634d3e856b015f4a7cf7cd
 Virtualization: vmware
Operating System: CentOS Linux 8 (Core)
 CPE OS Name: cpe:/o:centos:centos:8
   Kernel: Linux 4.18.0-193.14.2.el8_2.x86_64
 Architecture: x86-64
For Ubuntu 20.04 LTS
root@ubuntu20s1:~# hostnamectl
 Static hostname: ubuntu20s1
   Icon name: computer-vm
     Chassis: vm
   Machine ID: ea202235f9834d979436db10d89b1347
```

```
    Boot ID: 9bcbe4f6d00e407a81634dd61d74ff39
 Virtualization: vmware
Operating System: Ubuntu 20.04.1 LTS
  Kernel: Linux 5.4.0-47-generic
 Architecture: x86-64
```

You can also use the `lsb_release -d` command to check your OS. Unlike Ubuntu OS, CentOS does not have `lsb` pre-installed. You can install it by using the `yum install -y redhat-lsb` command or just running `lsb_release -d` and, when prompted, say Y to install .

```
For CentOS 8.1
[root@centos8s1 ~]# lsb_release -d
[root@centos8s1 ~]# yum install -y redhat-lsb
[root@centos8s1 ~]# lsb_release -d
Description:    CentOS Linux release 8.2.2004 (Core)
```

⑤ You can also use the `-sc` and `-a` options to view even more information about your Linux operating system.

```
For Ubuntu 20.04 LTS
root@ubuntu20s1:~# lsb_release -d
Description:    Ubuntu 20.04.1 LTS
root@ubuntu20s1:~# lsb_release -sc
focal
root@ubuntu20s1:~# lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.1 LTS
Release:        20.04
Codename:       focal
```

⑥ You can also use the `cat` command to output information under the `/etc/` directory. Notice that this works on CentOS, but *not* on Ubuntu, so the better way to view the system information is to use `cat` with a wildcard `*`. See the next exercise .

```
For CentOS 8.1
[root@centos8s1 ~]# cat /etc/centos-release
CentOS Linux release 8.2.2004 (Core)
[root@centos8s1 ~]# cat /etc/system-release
```

```
CentOS Linux release 8.2.2004 (Core)
```

To check your Linux's distro release version information, use the `cat /etc/*release` command . This `cat` command is a good Linux command to remember as it provides an abundance of information on your Linux operating system. This command works on both CentOS and Ubuntu Linux OS.

⑦

```
[root@centos8s1 ~]# cat /etc/*release
CentOS Linux release 8.2.2004 (Core)
NAME="CentOS Linux"
VERSION="8 (Core)"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="8"
PLATFORM_ID="platform:el8"
PRETTY_NAME="CentOS Linux 8 (Core)"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:centos:centos:8"
HOME_URL="https://www.centos.org/"
BUG_REPORT_URL="https://bugs.centos.org/"
CENTOS_MANTISBT_PROJECT="CentOS-8"
CENTOS_MANTISBT_PROJECT_VERSION="8"
REDHAT_SUPPORT_PRODUCT="centos"
REDHAT_SUPPORT_PRODUCT_VERSION="8"
CentOS Linux release 8.2.2004 (Core)
CentOS Linux release 8.2.2004 (Core)
```

Also, try to run the `cat /etc/os-release` command and check the difference.

Information on Linux: Use the netstat Command to Validate TCP/UDP Ports

To get you ready for IP services installation, make your Linux servers useful for our lab, and apply what you have learned in the production, you first have to know how to use some Linux network tools. All devices are running on some network segment and are connected on the network; hence, we have to be aware of which ports are opened or closed. In other words, which IP services are running and available to your users are part of server network security, which

can be part of network access control. Let's see how we can check network configurations on Linux systems .

First, to get the list of all network interfaces on your Linux server, use `netstat -i`.

```
[root@centos8s1 ~]# netstat -i
Kernel Interface table
```

①

Iface	MTU	RX-OK	RX-ERR	RX-DRP	RX-OVR	TX-OK	TX-ERR	TX-DRP	TX-OVR	Flg
ens160	1500	5601	0	0	0	2587	0	0	0	BMR
lo	65536	0	0	0	0	0	0	0	0	LRU
virbr0	1500	0	0	0	0	0	0	0	0	BMU

② To practice our `netstat` commands , let's install the nginx web services server on your CentOS server and start web services. After starting the Nginx web services, run the `netst -tulpn : grep :80` command to confirm that your CentOS server is listening on port 80 o the local interface .

For CentOS 8.1, install and start the nginx web server.

```
[root@centos8s1 ~]# yum install -y nginx
[root@centos8s1 ~]# whatis nginx
nginx (3pm)          - Perl interface to the nginx HTTP server API
nginx (8)            - "HTTP and reverse proxy server, mail proxy server"
[root@centos8s1 ~]# whereis nginx
nginx: /usr/sbin/nginx /usr/lib64/nginx /etc/nginx /usr/share/nginx
/usr/share/man/man3/nginx.3pm.gz /usr/share/man/man8/nginx.8.gz
[root@centos8s1 ~]# systemctl start nginx
[root@centos8s1 ~]# netstat -tulpn | grep :80
tcp     0      0 0.0.0.0:80       0.0.0.0:*        LISTEN      6046/nginx:
master
tcp6    0      0 :::80                 :::*        LISTEN      6046/nginx:
master
```

At this point, if you log into your CentOS desktop through VMware's main console, you will be able t open the Nginx home page in the Firefox web browser using `http://localhost` or `http://192.168.183.130`. See Figure **8-1**.

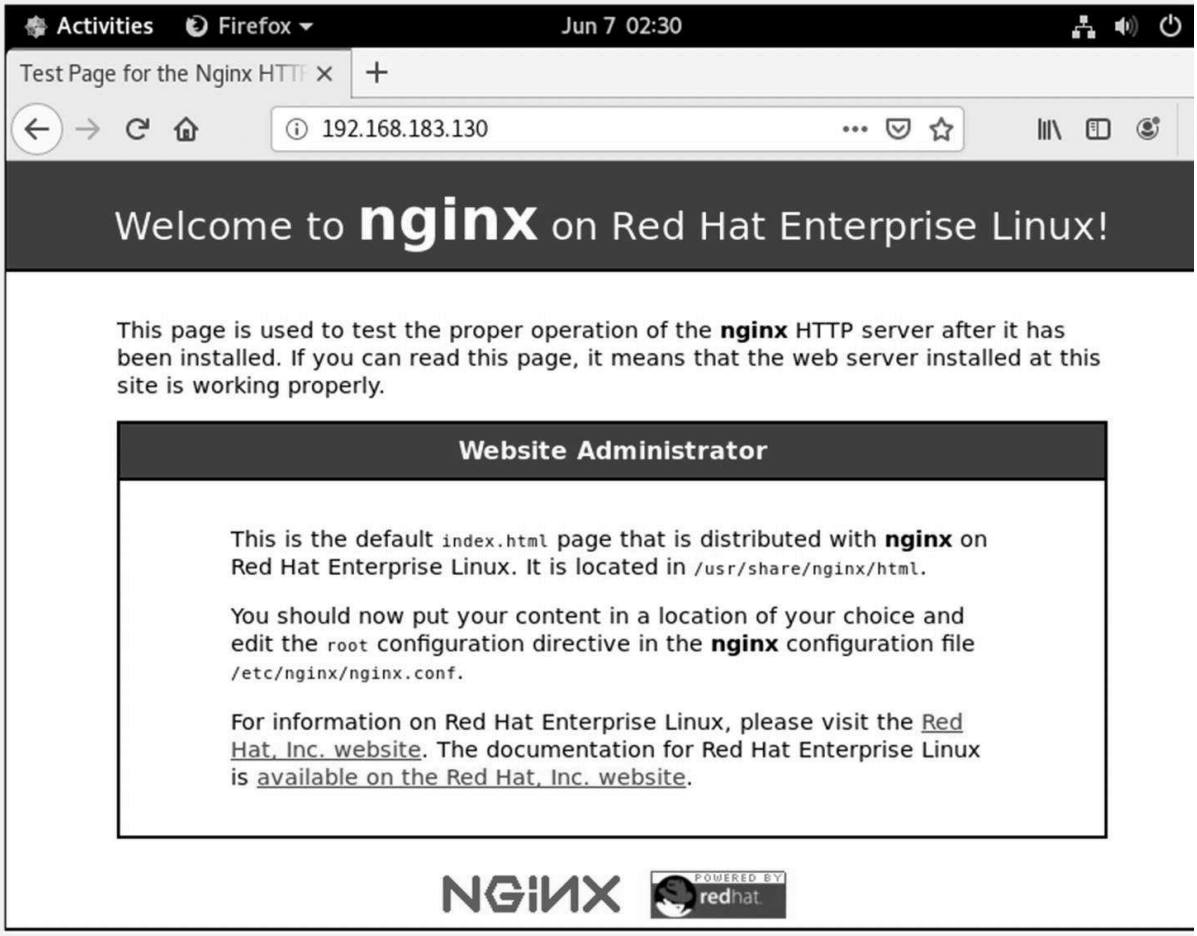


Figure 8-1. CenOS desktop , opening http://192.168.183.130

③ To permanently enable HTTP connections on port 80 and allow other machines to connect HTTP, add the HTTP service into the CentOS firewall's services list. Then verify that the HT firewall service is running correctly. To apply the changes, you must reload the firewall ser vice using the `firewall-cmd --reload` command . The full commands are listed here:

```
[root@centos8s1 ~]# firewall-cmd --permanent --add-service=http
success
[root@centos8s1 ~]# firewall-cmd --permanent --list-all
public
target: default
icmp-block-inversion: no
interfaces:
sources:
services: cockpit dhcpv6-client ftp http ntp ssh tftp
ports: 40000-41000/tcp
protocols:
masquerade: no
forward-ports:
```

```
source-ports:
icmp-blocks:
rich rules:
[root@centos8s1 ~]# firewall-cmd --reload
success
```

There is no DNS server in our network, so we have to use our CentOS server's IP address. W already know that our server IP is 192.168.183.130, but to verify this information, let's run following command. Optionally, you can use the `ip add` command and browse through the information.

```
[root@centos8s1 ~]# ip addr show ens160 | grep inet | awk '{ print $2; }' |
sed 's/\/.*$//'
192.168.183.130
fe80::f49c:7ff4:fb3f:bf32
[root@centos8s1 ~]# ip add
[...omitted for brevity]
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state U
group default qlen 1000
  link/ether 00:0c:29:85:a6:36 brd ff:ff:ff:ff:ff:ff
  inet 192.168.183.130/24 brd 192.168.183.255 scope global noprefixroute
ens160
  valid_lft forever preferred_lft forever
  inet6 fe80::f49c:7ff4:fb3f:bf32/64 scope link noprefixroute
  valid_lft forever preferred_lft forever
[...omitted for brevity]
```

At this point, you can open a web browser from your Windows 10 host PC/laptop and open the Ngi home page using `http://192.168.183.130`. Now you have successfully installed Nginx and open port 80 on your server. See Figure **8-2**.

Figure 8-2. Windows host, opening http://192.168.183.130

④ To list all the TCP or UDP ports on your Linux server, you can use the `netstat -a` comman the `-a` option stands for `all`. You can specify the `-t` option for TCP ports only or the `-u` opti for UDP ports in use.

For TCP:

```
[root@centos8s1 ~]# netstat -at
```

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	centos8s1:smux	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:hostmon	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:sunrpc	0.0.0.0:*	LISTEN

tcp	0	0	0.0.0.0:http	0.0.0.0:*	LISTEN
tcp	0	0	centos8s1:domain	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:ssh	0.0.0.0:*	LISTEN
tcp	0	0	centos8s1:ipp	0.0.0.0:*	LISTEN
tcp	0	64	centos8s1:ssh	192.168.183.1:54572	ESTABLISHE
tcp6	0	0	[::]:hostmon	[::]:*	LISTEN
tcp6	0	0	[::]:sunrpc	[::]:*	LISTEN
tcp6	0	0	[::]:http	[::]:*	LISTEN
tcp6	0	0	[::]:ftp	[::]:*	LISTEN
tcp6	0	0	[::]:ssh	[::]:*	LISTEN
tcp6	0	0	centos8s1:ipp	[::]:*	LISTEN

```
For UDP
[root@centos8s1 ~]# netstat -au
```

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
udp	0	0	0.0.0.0:48026	0.0.0.0:*	
udp	0	0	centos8s1:58365	_gateway:domain	ESTABLISHE
udp	0	0	centos8s1:domain	0.0.0.0:*	
udp	0	0	127.0.0.53:domain	0.0.0.0:*	
udp	0	0	0.0.0.0:bootps	0.0.0.0:*	
udp	0	0	centos8s1:bootpc	192.168.183.254:bootps	ESTABLISHE

udp	0	0	0.0.0.0:tftp	0.0.0.0:*
udp	0	0	0.0.0.0:sunrpc	0.0.0.0:*
udp	0	0	0.0.0.0:ntp	0.0.0.0:*
udp	0	0	0.0.0.0:snmp	0.0.0.0:*
udp	0	0	0.0.0.0:mdns	0.0.0.0:*
udp	0	0	0.0.0.0:hostmon	0.0.0.0:*
udp	0	0	centos8s1:323	0.0.0.0:*
udp6	0	0	[::]:tftp	[::]:*
udp6	0	0	[::]:sunrpc	[::]:*
udp6	0	0	[::]:mdns	[::]:*
udp6	0	0	[::]:hostmon	[::]:*
udp6	0	0	centos8s1:323	[::]:*
udp6	0	0	[::]:53619	[::]:*

⑤ If you want to list only the listening ports, then `netstat -l` can be used. The list was too lo so the following output only shows the top of the listening port information, but perhaps y can run this command on your CentOS VM and study the returned result .

```
[root@centos8s1 ~]# netstat -l
```

Active Internet connections (only servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	centos8s1:smux	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:hostmon	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:sunrpc	0.0.0.0:*	LISTEN

tcp	0	0	0.0.0.0:http	0.0.0.0:*	LISTEN
tcp	0	0	centos8s1:domain	0.0.0.0:*	LISTEN

[...omitted for brevity]

Like the `netstat -a` command, `netstat -l` also can be used with the `-t` or `-u` option for TCP- and UDP-only information. `netstat -lt` will only list TCP listening ports, whereas `netstat -lu` lists only UDP listening ports .

⑥

```
netstat -lt
netstat -lu
```

Run the previous commands on your Linux concole and see what output you receive from your CentOS server.

If you want to check which process uses a specific port, you can check it by typing `netstat -an` combined with a pipe (`|`) and `grep ':port_number'`. So if you want to check who is us SSH (port 22), you would issue the `netstat -an | grep ':22'` command.

⑦

```
netstat -an | grep ':22' <<< for SSH service
[root@centos8s1 ~]# netstat -an | grep ':22'
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0     64
192.168.183.130:22      192.168.183.1:54572     ESTABLISHED
tcp6       0      0 :::22
netstat -an | grep ':80'  <<< for http service
[root@centos8s1 ~]# netstat -an | grep ':80'
tcp        0      0 0.0.0.0:80              0.0.0.0:*           LISTEN
tcp6       0      0 :::80                   :::*                LISTEN
```

⑧ To check which IP network services are running on your server, you can issue the `netstat -ap` command. Here is an example of the `netstat -ap | grep ssh` command :

```
[root@centos8s1 ~]# netstat -ap | grep ssh
tcp   0      0 0.0.0.0:ssh     0.0.0.0:*           LISTEN      1257/sshd
tcp   0     64 centos8s1:ssh   192.168.183.1:54572  ESTABLISHED  1609/sshd:
root [pr
tcp6  0      0 [::]:ssh        [::]:*              LISTEN      1257/sshd
unix  2      [ ]         STREAM     CONNECTED     42663    1609/sshd: root
unix  2      [ ]         STREAM     CONNECTED     35797    1609/sshd: root
unix  2      [ ]         STREAM     CONNECTED     43450    2204/sshd: root@p
```

```
unix  2      [ ]         DGRAM                    43437    1609/sshd: root
unix  3      [ ]         STREAM     CONNECTED     43441    1609/sshd: root
unix  3      [ ]         STREAM     CONNECTED     43440    2204/sshd: root@p
unix  3      [ ]         STREAM     CONNECTED     32383    1257/sshd
unix  2      [ ]         STREAM     CONNECTED     32584    1257/sshd
```

⑨ Display the process ID (PID) and program names in your output; you can use the `netstat` command. It will display the protocol, IP service, and IP address information for your refer ence. The following command displays the `netstat -pt` output of an active SSH connectio

```
[root@centos8s1 ~]# netstat -pt
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address  Foreign Address    State       PID/Prograr
name
tcp   0    64 centos8s1:ssh   192.168.183.1:54572     ESTABLISHED 1609/sshd:
root [pr
```

⑩ The last set of commands consists of statistics commands ; since the output is quite long, yc can run the following commands from your CentOS VM console. Your clue to the `-s` handle the word *statistics*.

Please run the following commands from your CentOS Linux server to study the output at your own pace:

```
[root@centos8s1 ~]# netstat -s
[root@centos8s1 ~]# netstat -st
[root@centos8s1 ~]# netstat -su
```

Installing TFTP, FTP, SFTP, and NTP Servers

Here is a quick vocabulary lesson for non-CCNA readers:

FTP: File Transfer Protocol

SFTP: Secure File Transfer Protocol

TFTP: Trivial File Transfer Protocol

NTP: Network Time Protocol

In the world of business, PC end users and IT engineers alike use all kinds of network services supported within their network. Among the many IP services, the most popular network services used and supported by network and system engineers today include FTP, SFTP, TFTP, and NTP services. Each service can be installed on multiple distributed servers or even on networking platforms if the installation and configuration are supported. But in the lab, there is no reason to distribute network services across four separate virtual servers to support four different network services. Since we already have installed Nginx services, this makes five IP services on a single server. You can install all services on a single Linux server for testing purposes, but these services could be distributed across multiple servers in a real production environment.

In the production environment, FTP, SFTP, and TFTP are file services that require a large amount of storage space. The mass storage requirement means that these services are usually almost always installed on enterprise Linux or Windows servers, but not on networking devices such as routers and switches. NTP is a time service, so this service can run from Cisco routers. In NTP's case, the time synchronization service is provided so that both servers and network devices can run on standard time. All four of these IP services can be bundled into a single server and used as the IP services server in our lab for convenience. If you are coming from a pure networking background, you probably do not have a lot of experience building and supporting the Linux servers' enterprise IP services. Wouldn't it be great to have a better understanding of both the network and systems administration skill sets? Knowing more is freedom. Let's install FTP, SFTP, TFTP, and NTP servers on the CentOS 8 server for lab purposes. Remember, we are reserving the Ubuntu server as a Python + Docker server for later labs.

FTP Server Installation

File Transfer Protocol (FTP) is a secure and stable network service designed to send and receive many files between the server and clients. In other words, the simple advantage is that you can quickly send and receive files simultaneously compared to slower TFTP. FTP uses TCP port 21 by default and is less secure than its secure version, SFTP. There are several open source FTP servers available for Linux, for example, PureFTPd, ProFTPD, and vsftpd. In our example, we will install and use Very Secure

FTP daemon (`vsftpd`) as it is secure, stable, and fast. First, let's check if the FTP service is running on our CentOS 8 server.

The FTP server installation and verification in CentOS 8 are as follows:

① While installing the CentOS 8 virtual machine , we have selected to install the FTP server, but we do not know exactly which software version is installed. By default, CentOS will install `vsftpd` when you choose to install the FTP server from the installation media. Run the `vsftpd -version` command to check the current version of `vsftpd` on your CentOS server.

```
As expected, vsftpd version 3.0.3 is available.
[root@centos8s1 ~]# vsftpd -version
vsftpd: version 3.0.3
```

If you do not have the FTP software installed on your CentOS server, use the following two commands to complete your installation:

```
[root@centos8s1 ~]# sudo yum makecache
[root@centos8s1 ~]# yum install -y vsftpd
```

② Check if `vsftpd` is running on your server by typing `systemctl status vsftpd`.

```
[root@centos8s1 ~]# systemctl status vsftpd
● vsftpd.service - Vsftpd ftp daemon
 Loaded: loaded
(/usr/lib/systemd/system/vsftpd.service; enabled;
vendor preset: disabled)
 Active: active (running) since Mon 2021-01-04
19:15:34 AEDT; 1h 43min ago
 Process: 1238 ExecStart=/usr/sbin/vsftpd
/etc/vsftpd/vsftpd.conf (code=exited,
status=0/SUCCESS)
 Main PID: 1248 (vsftpd)
Tasks: 1 (limit: 11166)
Memory: 932.0K
CGroup: /system.slice/vsftpd.service
   ice248 /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf
Jan 04 19:15:33 centos8s1 systemd[1]: Starting Vsftpd
ftp daemon...
```

```
Jan 04 19:15:34 centos8s1 systemd[1]: Started Vsftpd
ftp daemon.
```

Use `Ctrl+C` to exit from the `vsftpd` status file.

③ You want the `vsftpd` daemon to start automatically during OS startup, so you must enable this service with the `systemctl enable` command . Once you have enabled the `vsftpd` service, use the same status command used previously to check the running status. When Active is `active (running)`, then the `vsftpd` service is active and working correctly.

```
[root@centos8s1 ~]# systemctl enable vsftpd --now
[root@centos8s1 ~]# systemctl status vsftpd
```

④ To use FTP for the lab, a few lines of the `vsftpd.conf` file need to be updated. Open the `vsftpd.conf` file in nano, check the existing configuration, and add a few configuration lines.

```
[root@centos8s1 ~]# nano /etc/vsftpd/vsftpd.conf
```

The following configuration should be ready out of the box, but configure the settings as follows if they are different:

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
```

To control the access to the FTP server using `user_list`, add the following lines after `userlist_enable=YES`:

```
userlist_file=/etc/vsftpd/user_list
userlist_deny=NO
```

To grant writable access to your user to its home directory, use this:

```
allow_writeable_chroot=YES
```

Also, `vsftpd` can use any port range for passive FTP connections. It is best practice to specify the port range for this use. Append the following configuration at the end of the `vsftpd.conf` file and save the file:

```
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=41000
```

To change where the files are uploaded and downloaded, update the `local_root` configuration. In this example, `$USER` will

take your user's ID and open FTP sessions from the `/home/pynetauto/ftp` directory.

```
user_sub_token=$USER
local_root=/home/$USER/ftp
```

After checking and updating the previous settings, append the following lines at the end of the `/etc/vsftpd/vsftpd.conf` file :

```
### ADDED by Admin ###
# Use user_list
userlist_file=/etc/vsftpd/user_list
userlist_deny=NO
# Allow writeable_chroot to the user
allow_writeable_chroot=YES
# Control passive port range to use between 40000-
41000
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=41000
# Use ftp directory under /home/user/
user_sub_token=$USER
local_root=/home/$USER/ftp
```

Remember the last line: `local_root = /home/$USER/ftp`. Make sure you create a local folder called `ftp` for the FTP access. We will be using the standard user to log into the FTP server, in my example, the `/home/pynetauto/ftp` directory.

```
[pynetauto@centos8s1 ~]$ mkdir ftp
```

⑤ Use the vi editor to change the FTP server configuration file so that all users can access it. Now add your user ID to the `user_list` file under `/etc/vsftpd`.

```
[root@centos8s1 ~]# vi /etc/vsftpd/user_list
# vsftpd userlist
# If userlist_deny=NO, only allow users in this file
# If userlist_deny=YES (default), never allow users
in this file, and
# do not even prompt for a password.
# Note that the default vsftpd pam config also checks
/etc/vsftpd/ftpusers
```

```
# for users that are denied.
root
bin
[...omitted for brivety]
nobody
pynetauto
~
"/etc/vsftpd/user_list" 21L, 371C
```

⑥ Once the changes have been saved to the file, open FTP and passive ports using the `firewall-cmd` commands . To allow firewall access to the FTP ports 20 and 21, run the following command:

```
firewall-cmd --add-service=ftp --permanent
(OR firewall-cmd --permanent --add-port=20-21/tcp)
firewall-cmd --permanent --add-port=40000-41000/tcp
setsebool -P ftpd_full_access on
firewall-cmd --reload
```

⑦ Make sure that `systemctl enable vsftpd` is issued to make `vsftpd` start automatically at the system startup. Also, here are the FTP troubleshooting commands for you to use. Use these commands as required to keep the service up and running.

```
systemctl enable vsftpd.service
systemctl start vsftpd.service
systemctl restart vsftpd.service
systemctl stop vsftpd.service
systemctl status vsftpd.service
```

We have learned `netstat` commands in the previous section, and you can use `netstat` commands to check that the FTP services are running smoothly.

```
netstat -ap | grep ftp
netstat -tupan | grep 21
netstat -na | grep tcp6
```

⑧ If the FTP service is working correctly, you should open your favorite web browser and open the FTP page using `ftp://SERVER_IP_ADDRESS` for your server. If you are prompted

with the user ID and password, enter your details. See Figure **8-3**.

`ftp://192.168.183.130/`

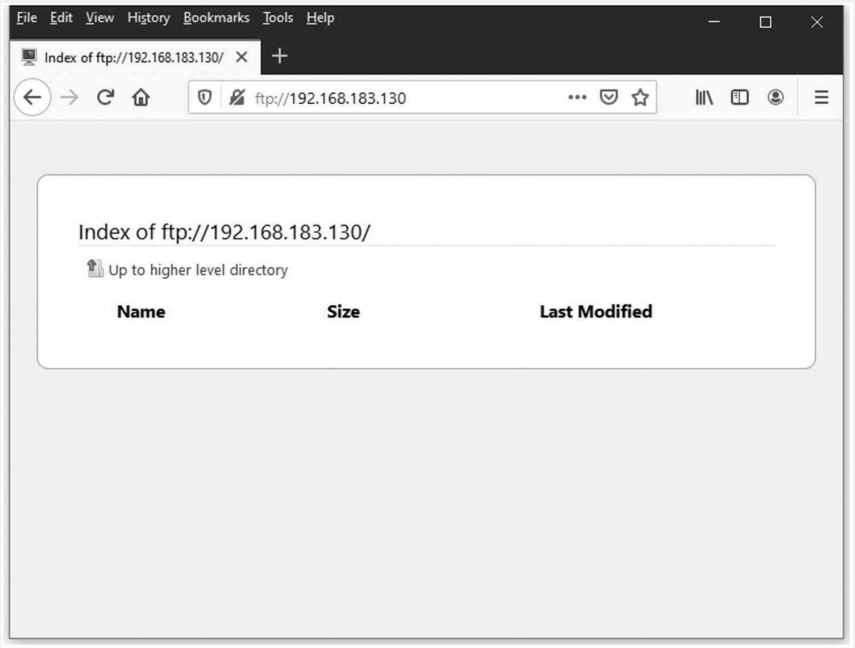


Figure 8-3. CentOS FTP server, FTP opened in Firefox web browser

⑨ You can also download the FileZilla client from your Windows host PC or Linux desktop and connect to your new FTP server using port 21.
URL: **`https://filezilla-project.org/download.php?platform=win64`**

Tip Enter `ftp://192.168.183.130` in the Host field. See Figure **8-4**.

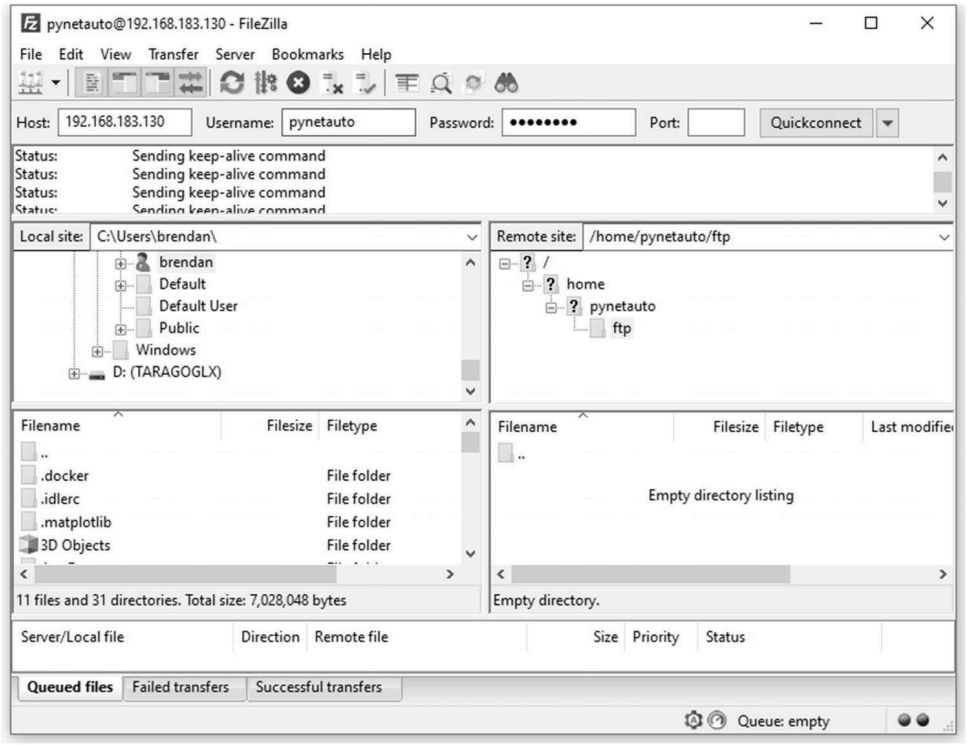


Figure 8-4. FileZilla client connected to FTP server

⑩ WinSCP is also another Windows application that comes in handy when connecting to your FTP server for file management. Upload or download between your host's Windows OS and CentOS 8 Linux server. See Figure 8-5 and Figure 8-6. You can download WinSCP for free from the following URL: URL: `https://winscp.net/eng/download.php`

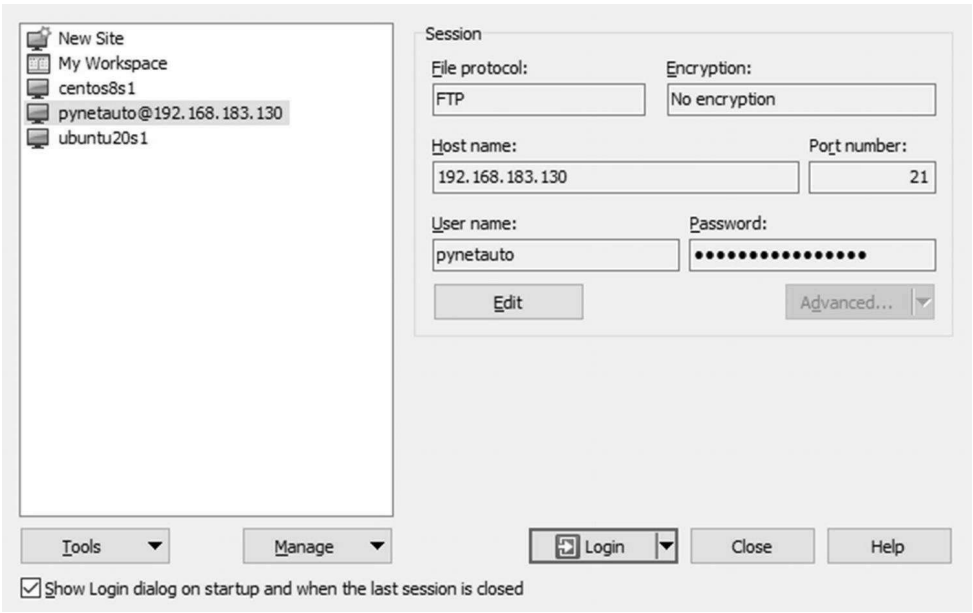


Figure 8-5. WinSCP, connecting to CentOS FTP server

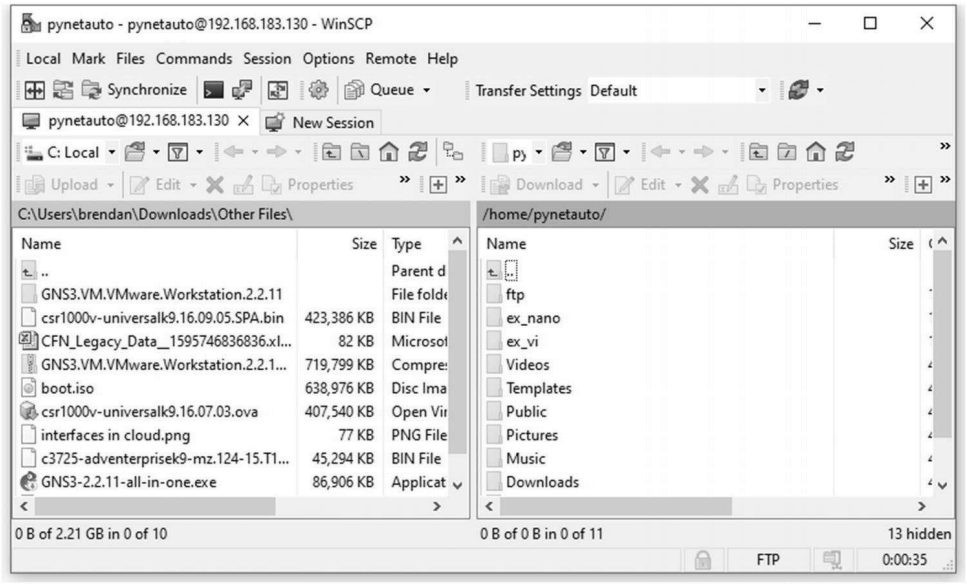


Figure 8-6. WinSCP, Connected to CentOS FTP server

You have completed the setup and FTP server login testing. You can now use the FTP server to share various files, such as IOS and configuration files, between the server and other network devices. Of course, you can also use FTP to back up router and switch configurations.

Use the `find` command to determine the directory or file location containing a file or directory name. In the following example, we are looking for the `pub` directory, which is usually the `ftp` default directory under the `./var/ftp/` directory.

```
[root @ localhost ~] # cd /
```

```
[root @ localhost /] # find . -name "pub"
```

Installing the SFTP Server

Secure File Transfer Protocol (SFTP) refers to the Secure FTP method to share files securely over the network. The difference between SFTP and FTP is whether the data sent across the network is encrypted or unencrypted. SFTP uses TCP/UDP port 22 as its default port. In most of the file transfer scenarios over the SFTP protocol, TCP 22 port will be used. UDP 22 was included in the TCP/IP development process at the developer's request but was not fully implemented. So, it is safe to say, SFTP mainly uses TCP port 22.

`vsftpd` was configured as FTP, but it is also an SFTP server. Since we already have `vsftpd` version 3.0.3 installed, we will configure this as an SFTP server too, running alongside the FTP server.

① First, log into CentOS 8 VM as the root user via the SSH client, PuTTY. You do not need to install new software for SFTP, but to begin, you need to create an SFTP-specific user account. Use `adduser` and `passwd` to create a new user for SFTP file transfer; the username created here is `sftpuser`.

```
[root@centos8s1 ~]# adduser sftpuser
[root@centos8s1 ~]# passwd sftpuser
Changing password for user sftpuser.
New password: **********
Retype new password: **********
passwd: all authentication tokens updated
successfully.
```

A quick tip: if you want to remove and re-create a user, you can use `userdel` command. To delete the user's home directory and mail pool, add `-r`.

```
[root@centos8s1 ~]# userdel -r sftpuser
```

② Create a directory `sftp` under the `/var/` directory and then create another child directory `sftpdata` under the `/var/sftp/` directory, where you will store and share your files. The directory name given here is `sftpdata`.

```
[root@centos8s1 ~]# mkdir -p /var/sftp/
[root@centos8s1 ~]# mkdir -p /var/sftp/sftpdata
```

③ Change the ownership of the `sftp` directory so that `sftpuser` can change the contents of `/var/sftp`.
Make the root user become the owner of the `/var/sftp/` directory. Change the permissions to 755 so that `sftpuser` can modify directories and files. Make the `sftpuser` user the owner of the `/var/sftp/sftpdata` directory.

```
[root@centos8s1 ~]# chown root:root /var/sftp
[root@centos8s1 ~]# chmod 755 /var/sftp
[root@centos8s1 ~]# chown sftpuser:sftpuser
/var/sftp/sftpdata
```

④ To restrict SSH access to the `/var/sftp/sftpdata` directory, you have to modify the `sshd_config` file in the `/etc/ssh/` directory. Open the `sshd_config` file to add/modify the following configuration. Copy the contents specified here to the bottom of the configuration file and save it .

```
[root@centos8s1 ~]# vi /etc/ssh/sshd_config
# Add the following to the end of the file.
### ADDED by Admin ###
Match User sftpuser
ForceCommand internal-sftp
PasswordAuthentication yes
ChrootDirectory /var/sftp
PermitTunnel no
AllowAgentForwarding no
AllowTcpForwarding no
X11Forwarding no
```

⑤ Restart the `sshd` service for the previous change to take effect.

```
[root@centos8s1 ~]# systemctl restart sshd
```

⑥ If you try to SSH into the CentOS server using the `sftpuser` account, the connection will be refused and disconnected. Even though SSH and SFTP use the same connection port, the `sftpuser` account can only be used for SFTP connections but not SSH connections. Give it a try .

```
[root@centos8s1 ~]# ssh sftpuser@192.168.183.130
The authenticity of host '192.168.183.130
(192.168.183.130)' can't be established.
ECDSA key fingerprint is
SHA256:b17PV5polHEKIcl+cFUCGA1KHGcl7xJkw/jhTf0kfZY.
Are you sure you want to continue connecting
(yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.183.130' (ECDSA)
to the list of known hosts.
sftpuser@192.168.183.130's password: ********
This service allows sftp connections only.
Connection to 192.168.183.130 closed.
```

This time, try to connect to the SFTP server using the sftp command. Enter the password to login, then use Ctrl+Z to disconnect from the SFTP server.

⑦

```
[root@centos8s1 ~]# sftp sftpuser@192.168.183.130
sftpuser@192.168.183.130's password: ********
Connected to sftpuser@192.168.183.130.
sftp>
```

Press Ctrl+Z to disconnect from the sftp server.

```
[1]+  Stopped                 sftp
sftpuser@192.168.183.130
```

Use the following commands to check if port 22 and `sshd` are working correctly (see Figure 8-7):

⑧

```
[root@centos8s1 ~]# netstat -ap | grep sshd
[root@centos8s1 ~]# netstat -tupan | grep 22
[root@centos8s1 ~]# netstat -na | grep tcp6
```

```
[root@centos8s1 ~]# netstat -tupan | grep 22
tcp        0      0 192.168.122.1:53        0.0.0.0:*               LISTEN
2083/dnsmasq
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
7681/sshd
tcp        0      0 192.168.183.130:48476   192.168.183.130:22      ESTABLISHED
7811/ssh
tcp        0      0 192.168.183.130:22      192.168.183.1:54798     ESTABLISHED
3215/sshd: root [pr
tcp        0     64 192.168.183.130:22      192.168.183.1:56444     ESTABLISHED
7417/sshd: root [pr
tcp        0      0 192.168.183.130:22      192.168.183.130:48476   ESTABLISHED
7812/sshd: sftpuser
tcp6       0      0 :::22                   :::*                    LISTEN
7681/sshd
udp        0      0 192.168.122.1:53        0.0.0.0:*
2083/dnsmasq
[root@centos8s1 ~]#
```

Figure 8-7. CentOS, netstat -tupan | grep 22 example

⑨ Finally, verify that your SFTP server is working correctly. You can use Windows applications such as FileZilla or WinSCP with user ID `sftpuser`, your password, and connecting port 22. See Figure 8-8 and Figure 8-9.

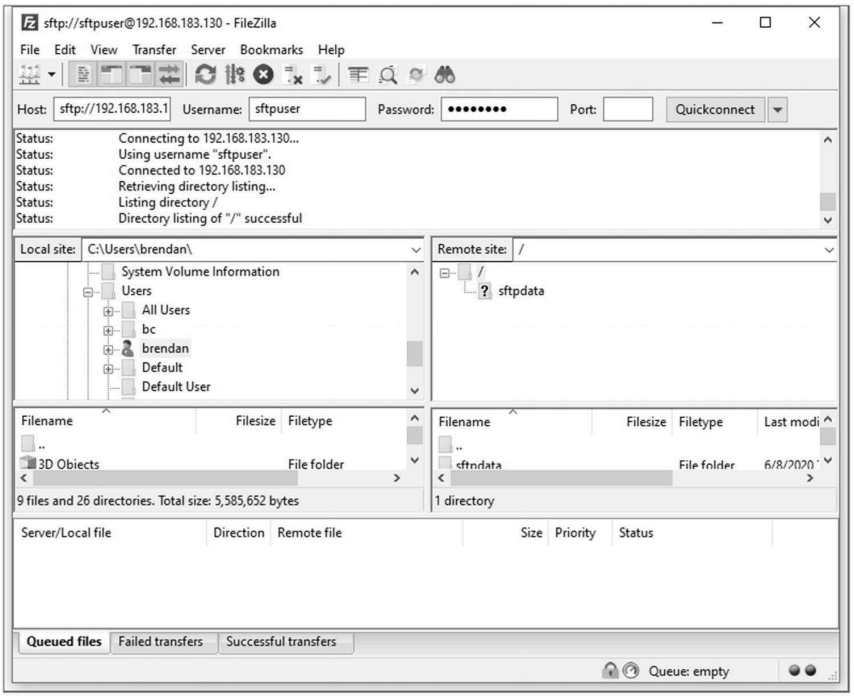


Figure 8-8. FileZilla client, SFTP connection example

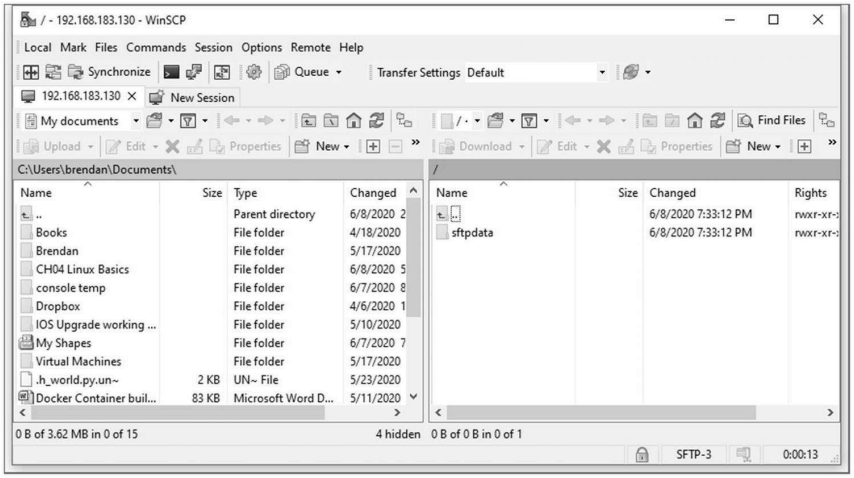


Figure 8-9. WinSCP, SFTP connection example

How do you print out all the usernames configured on your Linux server?

Just run the following command:

```
[root@centos8s1 ~]# awk -F ':' '{print $ 1}' /etc/passwd
```

```
root
```

```
bin
```

```
daemon
```

```
adm
```

```
[... omitted for brevity]
```

```
tcpdump
```

```
pynetauto
```

```
nginx
```

```
sftpuser
```

```
tftpuser
```

Now your CentOS server is running SFTP services for secure file sharing over the network. Whenever possible, SFTP must be used over FTP or TFTP for secure file transfers between two nodes (devices). Next is the installation of TFTP and its configuration.

Installing the TFTP Server

Trivial File Transfer Protocol (TFTP) uses UDP 69 as its default communication port. Because TFTP uses UDP, it has a much lighter operation than FTP or SFTP for file transfers. TFTP has been one of the most common file-sharing methods used to send and receive Cisco IOS or NX-OS images or configuration files across an internal network. It is still in use in many networks, but it still uses plain text for communication, and file transfer over UDP does not always guarantee the complete file transfer or security.

As mentioned, in the real production environment, you will not run all FTP, SFTP, and TFTP services on the same server at once, but we are working in the lab environment to collapse all three servers into one. Installing TFTP on CentOS 8 is similar to installing `vsftpd`. Let's go ahead and install the TFTP server for our lab use. It is important to remember that you always learn more and take away more when you build the lab yourself.

First, log in to CentOS 8.1 using the root user account and SSH into the server using PuTTY. As soon as you have logged in, check the firewall status by running the `systemctl status firewalld` command .

```
[root@centos8s1 ~]# systemctl status firewalld
●ifirewalld.service - firewalld - dynamic firewall
daemon
 Loaded: loaded
(/usr/lib/systemd/system/firewalld.service; enabled;
① vendor preset: enabled)
 Active: active (running) since Mon 2021-01-04
19:15:30 AEDT; 3h 18min ago
  Docs: man:firewalld(1)
Main PID: 1111 (firewalld)
 Tasks: 3 (limit: 11166)
 Memory: 33.8M
 CGroup: /system.slice/firewalld.service
   ervicewausr/libexec/platform-python -s
/usr/sbin/firewalld --nofork --nopid
```

Change the server firewall setting using the following command to enable TFTP traffic to communicate with your server :

```
② [root@centos8s1 ~]# firewall-cmd --permanent --
zone=public --add-service=tftp
[root@centos8s1 ~]# firewall-cmd --reload
```

③ Use yum install command to install tftp-server, xinetd service daemon and tftp client package. Run the following set of commands to complete your installation on your CentOS server :

```
[root@centos8s1 ~]# yum install xinetd tftp-server
tftp
[root@centos8s1 ~]# systemctl enable xinetd tftp
[root@centos8s1 ~]# systemctl start xinetd tftp
[root@centos8s1 ~]# systemctl status xinetd
●ixinetd.service - Xinetd A Powerful Replacement For
Inetd
 Loaded: loaded
(/usr/lib/systemd/system/xinetd.service; enabled;
```

```
vendor preset: enabled)
 Active: active (running) since Mon 2021-01-04
19:15:34 AEDT; 3h 20min ago
  Docs: man:xinetd
    man:xinetd.conf
    man:xinetd.log
 Process: 1245 ExecStart=/usr/sbin/xinetd -stayalive
-pidfile /var/run/xinetd.pid (code=exited,
status=0/SUCCESS)
 Main PID: 1281 (xinetd)
  Tasks: 1 (limit: 11166)
 Memory: 1.3M
 CGroup: /system.slice/xinetd.service
    ice281 /usr/sbin/xinetd -stayalive -pidfile
/var/run/xinetd.pid
```

④ Now that we know the TFTP server service is running correctly, we want to create a user account named `tftpuser` to assign just enough privileges to do its job, which is file transfer over UDP port 69. The `tftp` account created here is for system use only, so you cannot use it as a user account .

```
[root@centos8s1 ~]# useradd -s /bin/false -r tftpuser
```

⑤ Create a directory called `tftpdir` under the `/var` directory. All future TFTP file-sharing will be through this directory.

```
[root@centos8s1 ~]# mkdir /var/tftpdir
```

⑥ Change the permissions of the `tftpdir` directory so the `tftp` system account has permissions to this directory. To allow some freedom, we'll modify the directory with `chmod 777` .

```
[root@centos8s1 ~]# chown tftpuser:tftpuser
/var/tftpdir
[root@centos8s1 ~]# chmod 777 /var/tftpdir
```

⑦ Next, create a service file called `tftp` in `/etc/xinetd.d/`, copy the following content, and save the file :

```
[root@centos8s1 ~]# nano /etc/xinetd.d/tftp
service tftp
```

```
{
  socket_type = dgram
  protocol = udp
  wait = yes
  user = root
  server = /usr/sbin/in.tftpd
  server_args = -c -s /var/tftpdir -v -v -v -u tft-
puser -p
  disable = no
  per_source = 11
  cps = 100 2
  flags = IPv4
}
```

⑧ For TFTP to work well within SELinux , we must update the change context (`chcon`) of our `tftp` directory file `tftpdir`.

```
[root@centos8s1 ~]# chcon -t tftpdir_rw_t
/var/tftpdir
```

Let's restart the `tftp` and `xinetd` services once more.

```
[root@centos8s1 ~]# systemctl restart xinetd tftp
```

⑨ Before performing the verification task for the TFTP server, let's check two configurations, which are important for the TFTP operation on the CentOS server. First, make sure that the `/etc/selinux/config` configuration is adjusted, and second, check the `setsebool` settings to allow `tftp` write and directory access.

First, save the file after changing `SELINUX = enforcing` to `SELINUX = permissive`. See Figure **8-10**.

```
[root@centos8s1 ~]# nano /etc/selinux/config
```

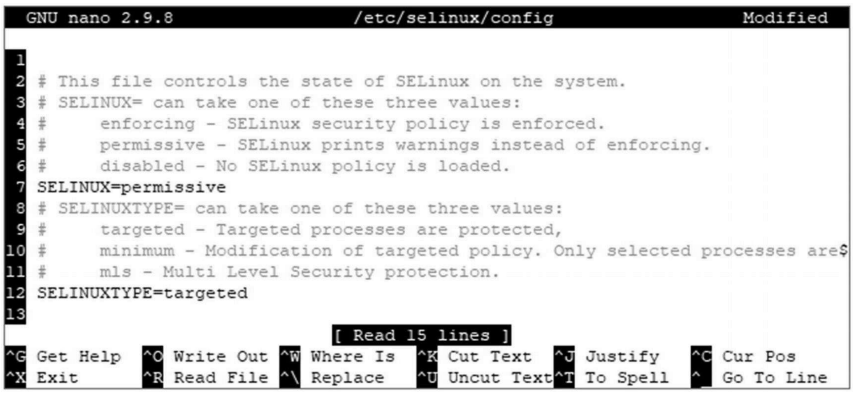


Figure 8-10. CentOS, /etc/selinux/config update

Second, we have to check the SELinux Boolean values of `tftp_anon_write` and `tftp_home_dir`, so SELinux allows TFTP file transfers. Check the `tftp` permissions using the following `getsebool` command if both `tftp_anon_write` and `tftp_home_dir` are showing as `off`. You must enable them using the subsequent commands.

```
[root@centos8s1 ~]# getsebool -a | grep tftp
tftp_anon_write --> off
tftp_home_dir --> off
```

Update the SELinux Boolean values of `tftp_anon_write` and `tftp_home_dir`.

```
[root@centos8s1 ~]# setsebool -P tftp_anon_write 1
[root@centos8s1 ~]# setsebool -P tftp_home_dir 1
[root@centos8s1 ~]# getsebool -a | grep tftp
tftp_anon_write --> on
tftp_home_dir --> on
```

Finally, give the last restart of the `xinetd` and `tftp` services on the CentOS server.

```
[root@centos8s1 ~]# systemctl restart xinetd tftp
```

⑩ On another Linux machine, the `ubuntu20s1` (192.168.183.129) server, install the `tftp` client using the `apt install -y tftp` command and perform the following verification tasks .

First, install the `tftp` client on the `ubuntu20s1` server.

```
root@ubuntu20s1:~# apt install tftp
```

Second, create `transfer_file01` and add some text as shown here:

```
root@ubuntu20s1:~# nano transfer_file01
```

```
This is a file transfer test file only.
Please contact the administrator if you have any
issues.
root@ubuntu20s1:~# cat transfer_file01
This is a file transfer test file only.
Please contact the administrator if you have any
issues.
```

Third, to test the `tftp` login to the CentOS `tftp` server, run `tftp 192.168.183.130`. Once connected, use the `put` command to upload `transfer_file01` to the TFTP server (192.168.183.130).

```
root@ubuntu20s1:~# tftp 192.168.183.130
tftp> put transfer_file01
Sent 99 bytes in 0.0 seconds
tftp>
```

Now go back to the `centos8s1` server and use the `ls /var/tftpdir` command to check that the file has been uploaded correctly.

```
[root@centos8s1 ~]# ls -lh /var/tftpdir
total 4.0K
-rw-rw-r--. 1 tftpuser  tftpuser    97 Jan  4 22:54
transfer_file01
```

Fourth, now the other way around, let's download a file from the TFTP server (`cenos8s1: 192.168.183.130`) to the file client (`ubuntu20s1: 192.168.183.132`) with the filename `transfer_file77`.

On the CentOS server, let's copy the `transfer_file01` file as `transfer_file77`.

```
[root@centos8s1 ~]# cp /var/tftpdir/transfer_file01
/var/tftpdir/transfer_file77
[root@centos8s1 ~]# ls /var/tftpdir
transfer_file01  transfer_file77
```

On the Ubuntu server , issue the `get file_name` command to download the `transfer_file05` file from the CentOS TFTP server.

```
root@ubuntu20s1:~# tftp 192.168.183.130
tftp> get transfer_file77
Received 99 bytes in 0.0 seconds
tftp> ^Z # Use Ctrl+Z to exit tftp mode
```

```
[1]+  Stopped                 tftp 192.168.183.130
root@ubuntu20s1:~# ls transfer*
transfer_file01  transfer_file77
```

You have successfully installed TFTP software on the CentOS 8 server and verified the TFTP operations between the server and a client. So far, FTP, SFTP, and TFTP are all up and running on the CentOS server, and next, you will quickly install the time services (NTP server) software on CentOS. When we test various scenarios in our lab or study or work, these servers will come in handy .

You can check if TFTP is working properly by using `net-tool`. If your CentOS VM does not recognize the `netstat` command, you can use `yum install -y net-tool` to install it and run the following set of commands to check the TFTP-related port operation:

```
netstat -na | grep udp6
```

```
netstat -lu
```

```
netstat -ap | grep tftp
```

```
netstat -tupan
```

```
netstat -tupan | grep 69
```

Installing the NTP Server

Network Time Protocol (NTP) is used to synchronize the time for enterprise devices such as servers, firewalls, routers, and switches. The NTP services use UDP port 123 and an IP network service that must be present in the corporate network to keep the time correct. The devices running in the same network segment or time zone need to have a consistent time, and the NTP server can provide this service to other devices. If hundreds or thousands of networked devices on a network all run on their hardware clock time, there will be no consistent timestamps for log and system files. NTP servers serve as a time aid to help the network equipment agree on a single referenced time. Let's go ahead and install the NTP server on the CentOS 8 server and perform a quick test run for our confirmation. On the previous version of CentOS, CentOS 7.5, NTP was available for installation, but on CentOS 8, the `chrony` daemon provides both NTP server and NTP client services.

First, use the SSH client to log into the CentOS 8 VM using the root user credentials. After you have logged in, check whether the server time and time zone are correct. If any of this information is incorrect, follow these directions to set your clock and time zone correctly:

1a. Check the time on the server.

```
[root@centos8s1 ~]# clock
2021-01-05 09:09:56.710758+11:00
```

1b. Check the time zone, NTP status, and other time details. [root@centos8s1 ~]# `timedatectl1c`. If your timezone is not correct, search the time zones list and find your country/city. Press Ctrl+Z to exit the list.

①

```
[root@centos8s1 ~]# timedatectl list-timezones
```

1d. Now set the correct time zone. If you are in another city and country, your time zone will be different mine, so adjust the time according to your geolocation .

```
[root@centos8s1 ~]# timedatectl set-timezone
Australia/Sydney
```

1e. Confirm that your server is running at the correct local time.

```
[root@centos8s1 ~]# clock
2021-01-05 09:16:05.039324+11:00
[root@centos8s1 ~]# date
Tue Jan  5 09:16:13 AEDT 2021
```

② Next, if `chrony` is not yet installed, install the NTP server and client using the `dnf` command, `dnf install chrony`.

```
[root@centos8s1 ~]# dnf install chrony
```

③ To make `chronyd` run at startup, enable the `chrony` services by typing `systemctl enable chronyd`.

```
[root@centos8s1 ~]# systemctl enable chronyd
```

④ Open the `chrony.conf` file under `/etc/chrony.conf` and allow your server and lab network. `chrony` will work as an NTP server for a local network after the subnet is allowed under the configuration file. Make sure you update the subnet to reflect your VM NAT subnet. Also, to make this time server more believable, we will change the local stratum to 3. Most of the network devices

and server will be happy to sync time with the NTP server with equal or less than stratum 5.

```
[root@centos8s1 ~]# vi /etc/chrony.conf
```

Update the following lines in bold in vi's `--INSERT--` mode and then save the change with the `:wq` command:

```
[...omitted for brevity]
# Allow NTP client access from local network.
#allow 192.168.0.0/16
# ADDED by pynetauto
allow 192.168.183.0/24    # Add this line to allow
local communication
# Serve time even if not synchronized to a time
source.
# local stratum 10
# ADDED by pynetauto
local stratum 5           # Add this line to make
this as a stratum 3 server
[...omitted for brevity]
:wq
```

Restart the `chronyd` service to make the changes take effect .

```
[root@centos8s1 ~]# systemctl stop chronyd
[root@centos8s1 ~]# systemctl start chronyd
[root@centos8s1 ~]# systemctl status chronyd
```

⑤ Open the firewall port to allow for incoming NTP requests on the network and reload the firewall .

```
[root@centos8s1 ~]# firewall-cmd --permanent --add-
service=ntp
success
[root@centos8s1 ~]# firewall-cmd --reload
Success
```

⑥ Finally, to apply the change, restart the `chronyd` daemon .

```
[root@centos8s1 ~]# systemctl restart chronyd
```

⑦ Go to your `Ubuntu 20 LTS` server, set the time zone, install `ntp-date`, and use `ntpdate 192.168.183.130` to synchronize time

with this server. If your NTP server IP is different, do not forget to update the IP address to reflect your NTP server IP .

7a. Update the time zone using the `timedatectl` commands.

Check the current time and date on your Ubuntu server.

```
root@ubuntu20s1:~# timedatectl
    Local time: Mon 2021-01-04 22:32:42 UTC
  Universal time: Mon 2021-01-04 22:32:42 UTC
    RTC time: Mon 2021-01-04 22:32:42
    Time zone: Etc/UTC (UTC, +0000)
System clock synchronized: yes
    NTP service: active
  RTC in local TZ: no
```

Update the time zone and check the time and date one more time.

```
root@ubuntu20s1:~# timedatectl set-timezone
Australia/Sydney
root@ubuntu20s1:~# timedatectl
    Local time: Tue 2021-01-05 09:33:29 AEDT
  Universal time: Mon 2021-01-04 22:33:29 UTC
    RTC time: Mon 2021-01-04 22:33:29
    Time zone: Australia/Sydney (AEDT, +1100)
System clock synchronized: yes
    NTP service: active
  RTC in local TZ: no
```

7b. Check the date, hardware clock, and time zone .

```
root@ubuntu20s1:~# date
Tue 05 Jan 2021 09:35:23 AM AEDT
root@ubuntu20s1:~# hwclock --show
2021-01-05 09:35:33.118208+11:00
root@ubuntu20s1:~# cat /etc/timezone
Australia/Sydney
```

7c. Install `ntpdate` on your Ubuntu server .

```
root@ubuntu20s1:~# apt-get install ntpdate
```

Notice the use of `apt-get` here. apt (Advanced Package Tool) merges the functionalities of `apt-get` and `apt-cache`, but for some packages, you still have to run `apt-get` to install cer-

tain packages. The `apt` command should work in most of the situations, but not always. There are plenty of articles on this difference on Google; you can do a quick lookup on this difference on Google.

URL: **https://askubuntu.com/questions/445384/what-is-the-difference-between-apt-and-apt-get**

URL: **https://phoenixnap.com/kb/apt-vs-apt-get**

7d. Synchronize the Ubuntu server time with the CentOS 8 NTP server time. If the NTP service is running correctly, the Ubuntu server will synchronize its clock and start referencing the NTP server for time requests .

```
root@ubuntu20s1:~# ntpdate -d 192.168.183.130
5 Jan 10:26:42 ntpdate[5667]: ntpdate
4.2.8p12@1.3728-o (1)
Looking for host 192.168.183.130 and service ntp
host found : 192.168.183.130
transmit(192.168.183.130)
receive(192.168.183.130)
[...omitted for brevity]
server 192.168.183.130, port 123
stratum 5, precision -26, leap 00, trust 000
[...omitted for brevity]
  0.000000 0.000000 0.000000 0.000000
delay 0.02592, dispersion 0.00006
offset 5.987462
5 Jan 10:26:58 ntpdate[5667]: step time server
192.168.183.130 offset 5.987462 sec
root@ubuntu20s1:~# date
Tue 05 Jan 2021 10:27:04 AM AEDT
```

Check the time on the NTP server (`centos8s1`). If time synchronization took place correctly as shown earlier, your NTP client's time will be synced to the NTP server's time.

```
[root@centos8s1 ~]# date
Tue Jan  5 10:27:10 AEDT 2021
```

Optionally, you can use the `ntpdate 192.168.183.130` or `ntpdate -u 192.168.183.130` command to update and resynchronize the time.

If you break this IP services server, now is the perfect time to take a snapshot of your CentOS server to preserve the server's working state. See Figure 8-11.

⑧

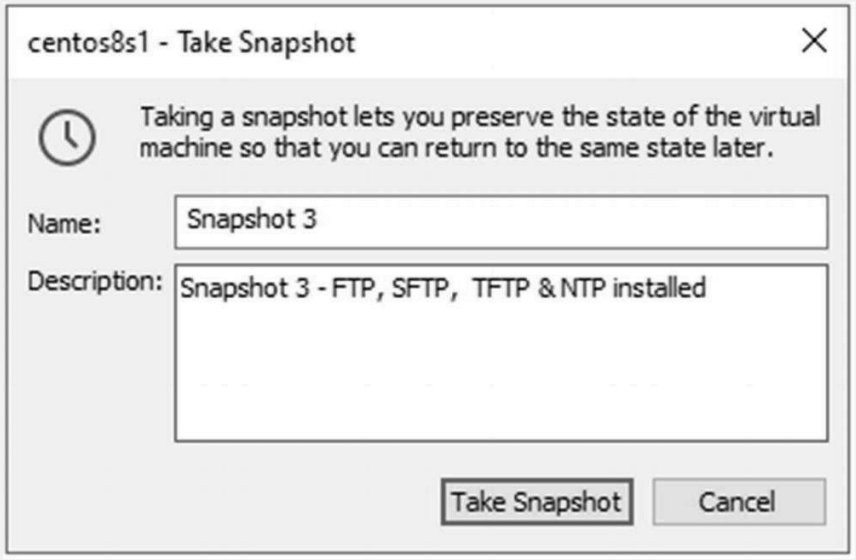


Figure 8-11. CentOS, taking a snapshot

You have successfully installed the NTP server, and this completes NTP, FTP, SFTP, and TFTP installation and verification. These servers will make your lab scenarios a lot more interesting, and also, you will be able to study, break, and rebuild your labs like your childhood favorite toys.

Linux TCP/IP Troubleshooting Exercise

This is the last exercise of this chapter. You will learn some Linux networking basic troubleshooting tips to test the TCP- and IP-related issues. When you encounter server networking issues, you can troubleshoot them yourself rather than palm them off to a Linux administrator.

① The first exercise is to check if your server allows ping (ICMP). Using `cat /proc/sys/net/ipv4/icmp_echo_ignore_all`, you can check if your server will respond to ICMP requests. The output is either 0 or 1. 0 indicates that ICMP is enabled and can respond; 1 indicates that ICMP is disabled, which means ICMP requests will be ignored.

For a CentOS server:

```
[root@centos8s1 ~]# cat /proc/sys/net/ipv4/icmp_echo_ignore_all
0
```

For a Ubuntu server:

```
root@ubuntu20s1:~# cat /proc/sys/net/ipv4/icmp_echo_ignore_all
```

```
0
```

For our testing purpose, let's update the Ubuntu server's value to 1 (from 0) so the ICMP response is disabled on the Ubuntu server. Then ping between two servers. After the testing, make sure you update this value back to 0.

```
root@ubuntu20s1:~# nano /proc/sys/net/ipv4/icmp_echo_ignore_all
root@ubuntu20s1:~# cat /proc/sys/net/ipv4/icmp_echo_ignore_all
1
[root@centos8s1 ~]# ping 192.168.183.130
root@ubuntu20s1:~# ping 192.168.183.130 -c 4
PING 192.168.183.130 (192.168.183.130) 56(84) bytes of data.
64 bytes from 192.168.183.130: icmp_seq=1 ttl=64 time=0.386 ms
64 bytes from 192.168.183.130: icmp_seq=2 ttl=64 time=0.602 ms
64 bytes from 192.168.183.130: icmp_seq=3 ttl=64 time=0.576 ms
64 bytes from 192.168.183.130: icmp_seq=4 ttl=64 time=0.607 ms
--- 192.168.183.130 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3053ms
rtt min/avg/max/mdev = 0.386/0.542/0.607/0.091 ms
```

As expected, there is no response from the Ubuntu server as ICMP requests from the CentOS server are ignored. If you ping the Ubuntu server from the CentOS server, 100 percent packet loss will result, as shown here :

```
[root@centos8s1 ~]# ping 192.168.183.132 -c 4
PING 192.168.183.132 (192.168.183.132) 56(84) bytes of data.
--- 192.168.183.132 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 79ms
```

② To test a server's open ports from a remote machine, you can use `telnet` + the IP address of remote server + the TCP port number. Telnet can be used for testing simple network socket connectivity but will only work on TCP ports (it does not work with UDP ports). For example, if you want to test the FTP (21) and SFTP (22) TCP connections from the Ubuntu server to the CentOS FTP/SFTP server, you can perform the following `telnet` test and confirm the end-to-end connection. After connected, to quit, type in `QUIT` and press Enter .
2a. Here is the FTP connection test:

```
root@ubuntu20s1:~# telnet 192.168.183.130 21
Trying 192.168.183.130...
Connected to 192.168.183.130.
Escape character is '^]'.
220 (vsFTPd 3.0.3)
QUIT
```

```
221 Goodbye.
Connection closed by foreign host.
```

2b. Here is the SSH/SFTP connection test :

```
root@ubuntu20s1:~# telnet 192.168.183.130 22
Trying 192.168.183.130...
Connected to 192.168.183.130.
Escape character is '^]'.
SSH-2.0-OpenSSH_8.0
QUIT
Invalid SSH identification string.
Connection closed by foreign host.
```

③ To see if a program or process is listening on a port, ready to accept a packet, use the `net-stat` command. `netstat` arguments (options) are listed here, so mix and match and try to run them on your Ubuntu server :

- `t`—show TCP port
- `u`—show UDP port
- `a`—show all
- `l`—show only listening processes
- `n`—do not resolve network IP address names or port numbers
- `p`—show process names listening on different ports

3a. Run the `netstat -tulnp` command from your Ubuntu server.

```
root@ubuntu20s1:~# netstat -tulnp
```

Here are the active Internet connections (only servers):

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	state	PID/Program name
tcp	0	0	127.0.0.1:45715	0.0.0.0:*	LISTEN	939/containerd
tcp	0	0	127.0.0.53:53	0.0.0.0:*	LISTEN	843/systemd-resolve
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN	1015/sshd: /usr/sbi
tcp	0	0	127.0.0.1:631	0.0.0.0:*	LISTEN	3251/cupsd

tcp6	0	0	:::22	:::*	LISTEN	1015/sshd: /usr/sbi
tcp6	0	0	::1:631	:::*	LISTEN	3251/cupsd
udp	0	0	127.0.0.53:53	0.0.0.0:*		843/systemd-resolve
udp	0	0	0.0.0.0:5353	0.0.0.0:*		859/avahi-daemon: r
udp	0	0	0.0.0.0:43413	0.0.0.0:*		859/avahi-daemon: r
udp	0	0	0.0.0.0:631	0.0.0.0:*		3252/cups-browsed
udp6	0	0	:::51143	:::*		859/avahi-daemon: r
udp6	0	0	:::5353	:::*		859/avahi-daemon: r

3b. Run the `netstat -tuna` command from your Ubuntu server.

```
root@ubuntu20s1:~# netstat -tuna
```

Here are the active Internet connections (servers and established):

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	127.0.0.1:45715	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.53:53	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:631	0.0.0.0:*	LISTEN
tcp	0	0	192.168.183.132:22	192.168.183.1:56036	ESTABLISHED

tcp6	0	0	:::22	:::*	LISTEN
tcp6	0	0	::1:631	:::*	LISTEN
udp	0	0	127.0.0.53:53	0.0.0.0:*	
udp	0	0	0.0.0.0:5353	0.0.0.0:*	
udp	0	0	0.0.0.0:43413	0.0.0.0:*	
udp	0	0	0.0.0.0:631	0.0.0.0:*	
udp6	0	0	:::51143	:::*	
udp6	0	0	:::5353	:::*	

You can also use `ss` command with arguments to see the listening ports or the port readiness. To see if a program or process is listening on a port, ready to accept a packet, use `ss`. The `ss` commands work on both Ubuntu and CentOS servers. See Figure **8-12**.

`t`—display TCP sockets

`u`—display UDP sockets

`l`—display listening sockets

`n`—do not resolve names

`p`—display process using socket

```
[root@centos8s1 ~]# ss -nutlp
```

OR

④

```
root@ubuntu20s1:~# ss -nutlp
```

```
root@ubuntu20s1:~# ss -nutlp
Netid State  Recv-Q Send-Q         Local Address:Port  Peer Address:PortProcess
udp   UNCONN 0      0                    0.0.0.0:631        0.0.0.0:*    users:(("cups-browsed",pid=4980,fd=7))
udp   UNCONN 0      0                    0.0.0.0:35619      0.0.0.0:*    users:(("avahi-daemon",pid=932,fd=14))
udp   UNCONN 0      0                127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=914,fd=12))
udp   UNCONN 0      0      192.168.183.129%ens33:68         0.0.0.0:*    users:(("systemd-network",pid=912,fd=19))
udp   UNCONN 0      0                    0.0.0.0:5353       0.0.0.0:*    users:(("avahi-daemon",pid=932,fd=12))
udp   UNCONN 0      0                       [::]:56859         [::]:*    users:(("avahi-daemon",pid=932,fd=15))
udp   UNCONN 0      0                       [::]:5353          [::]:*    users:(("avahi-daemon",pid=932,fd=13))
tcp   LISTEN 0      4096             127.0.0.53%lo:53         0.0.0.0:*    users:(("systemd-resolve",pid=914,fd=13))
tcp   LISTEN 0      128                  0.0.0.0:22         0.0.0.0:*    users:(("sshd",pid=1123,fd=3))
tcp   LISTEN 0      5                  127.0.0.1:631        0.0.0.0:*    users:(("cupsd",pid=4976,fd=7))
tcp   LISTEN 0      128                     [::]:22            [::]:*    users:(("sshd",pid=1123,fd=4))
tcp   LISTEN 0      5                     [::1]:631          [::]:*    users:(("cupsd",pid=4976,fd=6))
root@ubuntu20s1:~#
```

Figure 8-12. Ubuntu – ss –nutlp example

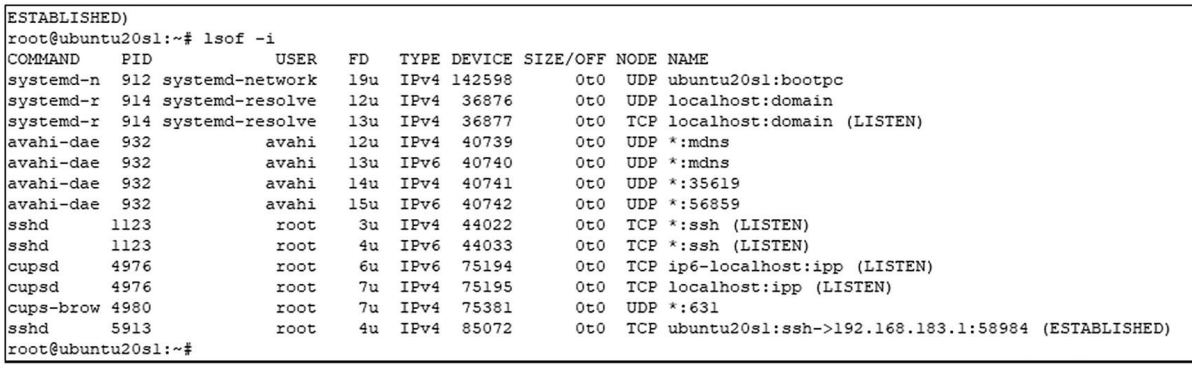
Use `lsof` to list the open ports on a Linux OS. To list the process name and PID numbers and open ports, type in `lsof -i`. You can use this command on most Linux OSs. See Figure **8-13**.

```
[root@centos8s1 ~]# lsof -i
```

OR

```
root@ubuntu20s1:~# lsof -i
```

⑤



```
ESTABLISHED)
root@ubuntu20s1:~# lsof -i
COMMAND    PID            USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
systemd-n  912 systemd-network   19u  IPv4 142598      0t0  UDP ubuntu20s1:bootpc
systemd-r  914 systemd-resolve   12u  IPv4  36876      0t0  UDP localhost:domain
systemd-r  914 systemd-resolve   13u  IPv4  36877      0t0  TCP localhost:domain (LISTEN)
avahi-dae  932           avahi   12u  IPv4  40739      0t0  UDP *:mdns
avahi-dae  932           avahi   13u  IPv6  40740      0t0  UDP *:mdns
avahi-dae  932           avahi   14u  IPv4  40741      0t0  UDP *:35619
avahi-dae  932           avahi   15u  IPv6  40742      0t0  UDP *:56859
sshd      1123            root    3u  IPv4  44022      0t0  TCP *:ssh (LISTEN)
sshd      1123            root    4u  IPv6  44033      0t0  TCP *:ssh (LISTEN)
cupsd     4976            root    6u  IPv6  75194      0t0  TCP ip6-localhost:ipp (LISTEN)
cupsd     4976            root    7u  IPv4  75195      0t0  TCP localhost:ipp (LISTEN)
cups-brow 4980            root    7u  IPv4  75381      0t0  UDP *:631
sshd      5913            root    4u  IPv4  85072      0t0  TCP ubuntu20s1:ssh->192.168.183.1:58984 (ESTABLISHED)
root@ubuntu20s1:~#
```

Figure 8-13. Ubuntu – lsof –i example

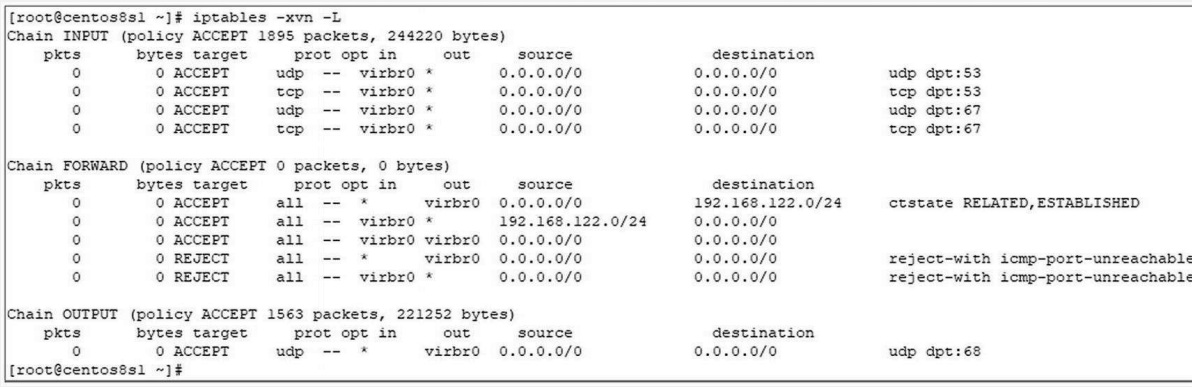
Use the `iptables` command to list more TCP IP communication information. Run `iptables -xvn -L` to display packets, interfaces, source and destination IP addresses, and ports in use. See Figure **8-14**.

```
root@ubuntu20s1:~# iptables -xvn -L
```

OR

```
[root@centos8s1 ~]# iptables -xvn -L
```

⑥



```
[root@centos8s1 ~]# iptables -xvn -L
Chain INPUT (policy ACCEPT 1895 packets, 244220 bytes)
    pkts      bytes target     prot opt in     out     source               destination
       0        0 ACCEPT     udp  --  virbr0 *       0.0.0.0/0            0.0.0.0/0           udp dpt:53
       0        0 ACCEPT     tcp  --  virbr0 *       0.0.0.0/0            0.0.0.0/0           tcp dpt:53
       0        0 ACCEPT     udp  --  virbr0 *       0.0.0.0/0            0.0.0.0/0           udp dpt:67
       0        0 ACCEPT     tcp  --  virbr0 *       0.0.0.0/0            0.0.0.0/0           tcp dpt:67

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
    pkts      bytes target     prot opt in     out     source               destination
       0        0 ACCEPT     all  --  *      virbr0  0.0.0.0/0            192.168.122.0/24    ctstate RELATED,ESTABLISHED
       0        0 ACCEPT     all  --  virbr0 *       192.168.122.0/24     0.0.0.0/0
       0        0 ACCEPT     all  --  virbr0 virbr0  0.0.0.0/0            0.0.0.0/0
       0        0 REJECT     all  --  *      virbr0  0.0.0.0/0            0.0.0.0/0           reject-with icmp-port-unreachable
       0        0 REJECT     all  --  virbr0 *       0.0.0.0/0            0.0.0.0/0           reject-with icmp-port-unreachable

Chain OUTPUT (policy ACCEPT 1563 packets, 221252 bytes)
    pkts      bytes target     prot opt in     out     source               destination
       0        0 ACCEPT     udp  --  *      virbr0  0.0.0.0/0            0.0.0.0/0           udp dpt:68
[root@centos8s1 ~]#
```

Figure 8-14. CentOS, iptables -xvn –L example

Perhaps you are connecting to your FTP server from a Windows client. Then how do you make an FTP connection to your FTP server from your Windows PC? You can use the Windows command-line prompt or Windows PowerShell. `Test-NetConnection` is a native PowerShell command and can be used to test simple connectivity such as FTP port 21. Let's start PowerShell from your Windows 10 host PC/laptop and quickly test if the FTP connection to the CentOS FTP server works correctly. See Figure **8-15**.

```
PS C:\Users\brendan> test-NetConnection -ComputerName 192.168.183.130 -Port 21
```

⑦

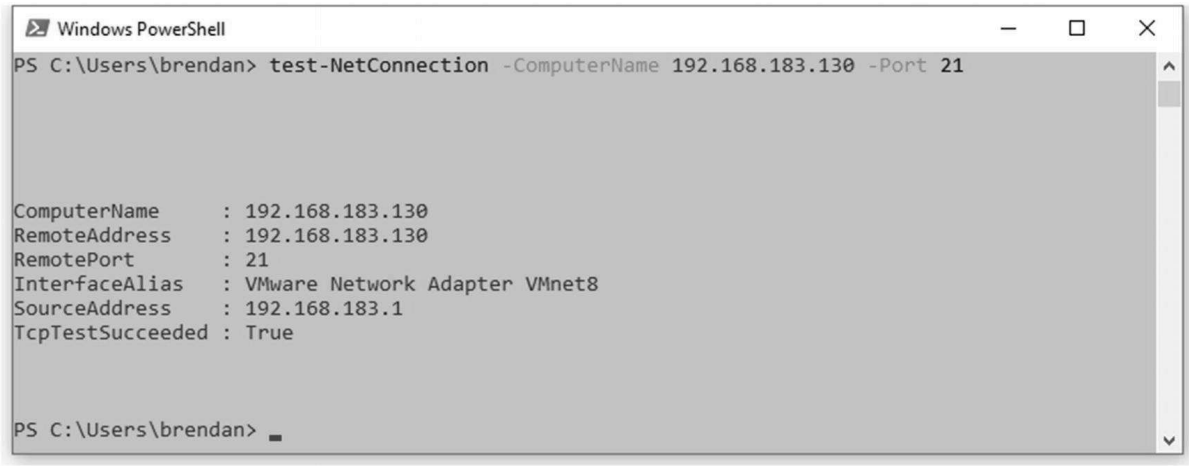


Figure 8-15. Windows 10 host PC, PowerShell test-NetConnection example

Summary

In this chapter, you learned some more Linux administration, starting with how to check system information and how to check TCP and UDP ports on the Linux systems. Then you were guided through IP services installation on a CentOS server to make an all-in-one lab server for TFTP, FTP, SFTP, and NTP services. This server will be used later for Python network automation labs in the second half of this book. Finally, you learned some basic TCP/IP and connection troubleshooting on Linux, so you can troubleshoot the connectivity issues on the move. I hope this was an interesting chapter for everyone. In the next chapter, Chapter 9, you will be tackling another challenging topic, regular expressions (a.k.a. regex). Understanding the inner workings of regex can boost your Python coding ability. You have to stay on course and complete the next chapter.

Storytime 4: Becoming a Savvy Linux Administrator, Perhaps Every IT Engineer's Dream?

Everyone in IT will agree that investing time and resources studying enterprise Linux is a worthwhile future investment while working in the IT industry. Becoming a savvy Linux administrator is many IT engineers' dream, if not every engineer. At least it has been my dream for some time. Can you remember your first Linux experience? Here are some common experiences Linux users may go through while trying to learn Linux, especially if you have been a Windows user all your life.

- Everyone said Ubuntu Desktop is the best Linux for beginners, so you downloaded the latest Linux `.iso` file and installed it on your PC. Within a few days, you wiped it with Windows 10, and you are back on the Windows OS like nothing ever happened.
- Dual-booting sounded like a cool thing to do, so you installed both Windows and Linux on the same host and tried the multibooting option on your laptop. A few weeks later, you noticed a sizable unused partition on your Windows Partition Manager and thought, what a waste of storage space. Then you remembered: it was the partition for your Linux operating system.
- As time went by, you were logging into Windows more and more, even though your PC had a multibooting option.
- Ubuntu or Gnome Desktop experience seemed like an OK user interface connected to the Internet, but you were still craving the Windows graphical user interface. You are hooked to the Windows GUI.
- Everyone said you have to study Linux commands, but no one told you how to study Linux commands. All the websites and videos said you have to learn them by heart, so you tried your best to memorize as much as you can, squeezing every command into your tiny brain, and the next day, you could not even remember a single command from the previous day's study.
- There is always some Linux guru at your work, and he is putting pressure on you to make sure you take on Linux again even though you left it for a few years. Never give up!

Perhaps you liked Linux from the get-go and enjoyed the experience of learning Linux. For some people, the Linux chapters could pose the first barrier in your Python network automation journey, but we must continue. Based on some quick Internet research, there is still only a small

percentage of true Linux administrators in the IT industry. There were three Linux engineers out of 80 IT engineers at my last workplace, and on my current team, there are only two members (out of 20) who are well-versed in Linux. Almost every engineer who can write programming code will agree that it is impossible to develop code and run it as a service only on Windows servers. If you are a Microsoft guru, it is possible that the blue screen of death appears a couple of times a year, and your best fix for Windows Server is to press the Reboot button. In other words, as a novice network automation engineer, we have to become familiar with Linux systems and start using Python on Linux. After all, Python and Linux go hand in hand. If you want to become good at Python or any coding, you must cover the Linux basics well. I agree, it is challenging to learn and master Linux administration, but we can always take one step at a time, and over time, we will get there slowly but surely. Still, I hope the contents in this chapter and the previous chapter will be enough to get you through this book and keep your interest in Linux alive. Since studying Python, I improved my game with Linux, and Linux has made me rethink my career and take a different IT career path. I highly recommend everyone take another crack at Linux. Let's keep trying!

Table of contents collapsed